Part 6 — Real-Time CI/CD Project

Jenkins Interview Notes | GitHub -> Jenkins -> Terraform -> AWS / EKS

This part explains how to build a real-world CI/CD pipeline that automatically checks code, validates Terraform, creates an execution plan, requests approval, and deploys infrastructure to AWS or Amazon EKS.

Each topic follows your five required headings:

- 1. Definition — What is it?
- 2. Purpose — Why do we use it?
- 3. Working — How does it work?
- 4. Practical Example — How do we use it in a project?
- 5. Scenario-Based Interview Questions and Answers — How can interviewers ask about it?

1. Real-Time CI/CD Project Architecture

1. Definition — What is it?

A CI/CD project automates the process of taking changes from a source-code repository and delivering them to a target environment.

- CI — Continuous Integration: Automatically checks code changes.
- CD — Continuous Delivery or Continuous Deployment: Automates the preparation or deployment of approved changes.
- Terraform: Infrastructure as Code (IaC) tool used to provision infrastructure.
- AWS: Cloud platform that provides infrastructure and managed services.
- EKS: Amazon Elastic Kubernetes Service, a managed Kubernetes service.

2. Purpose — Why do we use it?

- To reduce manual deployment work.
- To detect configuration errors before deployment.
- To make deployments repeatable and traceable.
- To reduce human errors and improve delivery speed.
- To apply approval and security controls before production changes.
- To maintain consistent infrastructure across environments.

3. Working — How does it work?

1. **GitHub** — Developer pushes code or opens a pull request
2. **Jenkins** — Webhook or polling triggers the pipeline
3. **Build and Validate** — Checkout, format, validate, tests and security checks
4. **Terraform Plan** — Review proposed infrastructure changes
5. **Manual Approval** — Authorized person approves production changes
6. **Terraform Apply -> AWS / EKS** — Provision or update infrastructure and verify the result

4. Practical Example — How do we use it in a project?

Imagine you update eks-cluster.tf in your EKS Terraform repository.

1. Push your change to GitHub.
2. GitHub triggers Jenkins.

3. Jenkins checks out the intended branch and commit.
4. Terraform initializes the backend and providers.
5. Jenkins runs formatting checks and validation.
6. Terraform generates a plan showing the proposed changes.
7. An authorized person reviews and approves the plan.
8. Terraform applies the approved changes to AWS.
9. Jenkins records the result and reports failures.

For an application deployment to EKS, you would usually add a separate application delivery stage to build and scan a container image, push it to Amazon ECR, and deploy the image to Kubernetes.

5. Scenario-Based Interview Questions and Answers

Q1. Explain your real-time CI/CD project in an interview.

Answer:

“My project automates AWS infrastructure provisioning using GitHub, Jenkins, and Terraform. When code is pushed to the repository, Jenkins checks out the code, initializes Terraform, runs formatting and validation checks, and generates a plan. After reviewing and approving the proposed changes, the pipeline applies the infrastructure changes to AWS. For EKS workloads, application images can be built, pushed to ECR, and deployed to the cluster through an additional deployment stage.”

Q2. Why do you use Jenkins instead of running Terraform manually?

Answer: Jenkins provides a repeatable workflow, centralized logs, automated validation, controlled approvals, credential integration, and a record of which build performed a change. Terraform can still be run manually when appropriate, but automation reduces routine human error.

Q3. Does Terraform deploy the application inside EKS automatically?

Answer: Not necessarily. Terraform can provision the EKS cluster and related infrastructure. Application deployment may be handled by Kubernetes manifests, Helm, or another deployment mechanism, although Terraform can manage some Kubernetes resources when configured with the appropriate provider.

2. GitHub Integration and Webhooks

1. Definition — What is it?

GitHub integration connects the source-code repository to Jenkins so that Jenkins can retrieve code and run automated pipelines.

A webhook is an HTTP notification sent by GitHub to Jenkins when a configured event occurs, such as a push or pull request.

2. Purpose — Why do we use it?

- To trigger builds automatically when code changes.
- To avoid manually starting every build.
- To validate changes before merging them.
- To maintain a record of the source commit used by a build.
- To identify problems early in the development process.

3. Working — How does it work?

1. Store your Jenkinsfile in the GitHub repository.
2. Configure a Jenkins Pipeline job with the repository URL and branch.

3. Configure GitHub webhook delivery or another supported build trigger.
4. Push a change or open a pull request.
5. Jenkins checks out the configured commit and runs the pipeline.

A webhook requires a reachable Jenkins endpoint and correct network, webhook, and authentication configuration. A local Jenkins server may need a secure, reachable webhook endpoint; alternatively, polling can be used where suitable.

4. Practical Example — How do we use it in a project?

Suppose your repository is e2e-k8s-project1, with Terraform files under EKS-TF/.

A simplified Jenkins checkout stage is:

```
stage('Checkout') {
    steps {
        dir('EKS-TF') {
            checkout scm
            bat 'git rev-parse HEAD'
        }
    }
}
```

The git rev-parse HEAD command records the checked-out commit hash in the console output. The dir path must match the repository's actual folder structure.

For a multibranch pipeline, Jenkins can discover branches and pull requests based on the configured source-code management integration.

5. Scenario-Based Interview Questions and Answers

Q1. You push code to GitHub, but Jenkins does not start a build. How do you troubleshoot it?

Answer:

1. Confirm that the push reached the expected repository and branch.
2. Check whether the Jenkins job monitors that branch.
3. Review GitHub webhook delivery history for HTTP errors.
4. Verify Jenkins endpoint reachability and the configured trigger.
5. Check Jenkins system logs and job configuration.
6. Run a manual build to isolate whether the problem is the trigger or the pipeline.

Q2. How do you ensure Jenkins deploys the correct version of Terraform code?

Answer: I pin the pipeline to the intended branch or commit, record the commit hash, and review the Terraform plan generated from that exact checkout. I also prevent unreviewed feature-branch changes from reaching production.

Q3. Should every pull request automatically deploy to production?

Answer: No. Pull requests should normally trigger validation, tests, and possibly a plan. Production deployment should follow the repository's merge rules, authorization requirements, and approval process.

3. Jenkinsfile and Pipeline Stages

1. Definition — What is it?

A Jenkinsfile is a text file that defines the pipeline's stages, steps, environment, parameters, conditions, and post-build actions.

A stage represents a major phase of the pipeline, such as Checkout, Validate, Plan, Approval, or Deploy.

2. Purpose — Why do we use it?

- To keep pipeline logic in version control.
- To make builds repeatable and reviewable.
- To divide complex deployment processes into understandable stages.
- To identify exactly where a pipeline failed.
- To support collaboration and change review.

3. Working — How does it work?

Jenkins reads the Jenkinsfile and executes its stages in the defined order, subject to conditions and parallel branches. If a stage fails, later stages are generally skipped unless the pipeline explicitly handles the failure differently.

A Declarative Pipeline commonly uses:

- pipeline: Defines the pipeline.
- agent: Selects where it runs.
- parameters: Defines user inputs.
- environment: Defines environment variables.
- stages: Contains the pipeline stages.
- steps: Contains the commands and operations.
- post: Defines actions after execution.

4. Practical Example — How do we use it in a project?

A typical Terraform Jenkinsfile may have these stages:

```
stages {
    stage('Checkout') {
        steps {
            checkout scm
        }
    }

    stage('Terraform Init') {
        steps {
            bat 'terraform init'
        }
    }

    stage('Terraform Validate') {
        steps {
            bat 'terraform fmt -check -recursive'
            bat 'terraform validate'
        }
    }

    stage('Terraform Plan') {
        steps {
            bat 'terraform plan -out=tfplan'
        }
    }
}
```

This is a simplified sequence. It assumes a Windows agent and a Terraform working directory. Add approval and apply stages only after implementing the appropriate permissions and deployment controls.

5. Scenario-Based Interview Questions and Answers

Q1. Your Jenkins pipeline fails during Terraform validation. Will the apply stage run?

Answer: In a normal sequential Declarative Pipeline, a validation failure stops subsequent stages from running. I would inspect the validation error, fix the configuration, and rerun the pipeline.

Q2. Why do you keep the Jenkinsfile in GitHub?

Answer: Pipeline changes can be reviewed, versioned, and rolled back. It also allows the project and its deployment workflow to evolve together.

Q3. How do you separate development and production deployment?

Answer: I use appropriate branches or protected deployment jobs, environment-specific configuration, separate access controls, and production approval gates. I avoid relying only on a user-selectable parameter to protect production.

4. Build and Code Quality Checks

1. Definition — What is it?

The build and quality stage checks whether the code is syntactically correct, formatted consistently, and ready for subsequent testing or deployment.

For Terraform projects, this includes formatting checks, configuration validation, and optionally security scanning and automated tests.

2. Purpose — Why do we use it?

- To catch mistakes before modifying cloud infrastructure.
- To standardize code formatting.
- To prevent obvious configuration errors from reaching production.
- To identify security risks in infrastructure configuration.
- To improve the reliability of later deployment stages.

3. Working — How does it work?

A common order is:

1. Check out the intended code.
2. Initialize Terraform and download the required providers.
3. Run formatting checks.
4. Validate Terraform configuration.
5. Run additional linting or security scans.
6. Continue to planning only if the required checks pass.

Remember that terraform validate checks configuration correctness, not whether the AWS account has sufficient permissions or whether every planned change is safe.

4. Practical Example — How do we use it in a project?

Run these commands from the Terraform root directory:

```
terraform init -input=false
terraform fmt -check -recursive
terraform validate
```

For your EKS project, the root directory might be EKS-TF. If the project uses a remote backend, terraform init must be configured with the correct backend and credentials.

You can add security scanning tools such as Checkov or Trivy's configuration scanning capabilities if they fit your team's tooling and policy.

5. Scenario-Based Interview Questions and Answers

Q1. terraform validate passes, but terraform plan fails. Why?

Answer: Validation checks the configuration's internal consistency. Planning also needs to evaluate variables, provider configuration, data sources, backend state, and cloud access. I would inspect the plan error and determine which dependency or permission is missing.

Q2. Should the pipeline automatically run terraform fmt and change code?

Answer: In a developer workflow, automatic formatting can be useful. In a CI validation job, I often use terraform fmt -check -recursive so that the pipeline detects formatting problems without silently modifying the checked-out code.

Q3. How do you catch insecure Terraform code before deployment?

Answer: I add static configuration scanning and policy checks, review IAM permissions and network exposure, and fail the pipeline when defined security policies are violated.

5. Terraform Backend, State, and Initialization

1. Definition — What is it?

Terraform initialization prepares a working directory for Terraform operations. It downloads providers and modules and configures the backend used to store Terraform state.

Terraform state records Terraform's understanding of managed infrastructure. A remote backend allows multiple authorized users or pipelines to work with shared state.

2. Purpose — Why do we use it?

- To install the correct providers and modules.
- To connect the pipeline to the intended state backend.
- To coordinate infrastructure changes between users and automation.
- To avoid independent local state files that can become inconsistent.
- To reduce the risk of concurrent Terraform operations.

3. Working — How does it work?

1. Jenkins checks out the Terraform repository.
2. Terraform reads the backend configuration.
3. Terraform initializes providers and modules.
4. It connects to the configured remote backend.
5. During planning or applying, Terraform reads and updates state as required.
6. Backend locking prevents conflicting operations when supported and correctly configured.

For AWS, a common setup uses an S3 bucket for state storage and a supported locking mechanism. Terraform versions supporting S3-native state locking can use use_lockfile = true; DynamoDB-based locking is deprecated in newer Terraform versions. Existing projects should follow the documentation for their installed Terraform version and backend configuration.

4. Practical Example — How do we use it in a project?

An illustrative S3 backend configuration is:

```
terraform {
```

```
  backend "s3" {
    bucket       = "s3buckk8sproject1"
    key          = "eks/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

This example assumes the bucket exists in us-east-1 and the installed Terraform version supports S3 lock files. The identity running Jenkins must have the appropriate backend permissions.

Keep state in a dedicated, access-controlled bucket. Do not commit terraform.tfstate or state backups to GitHub.

5. Scenario-Based Interview Questions and Answers

Q1. Two Jenkins builds attempt to apply Terraform at the same time. What could happen?

Answer: Concurrent operations can conflict and potentially produce inconsistent changes. I would use a backend locking mechanism and prevent overlapping pipeline executions where appropriate. Locking and pipeline concurrency controls complement one another.

Q2. Jenkins reports that the Terraform state cannot be accessed. What do you check?

Answer: I check the bucket name and region, state key, AWS identity, IAM permissions, bucket policy, encryption configuration, network connectivity, and backend initialization. I also check whether a state lock is held by another operation.

Q3. Why should Terraform state not be committed to GitHub?

Answer: State can contain sensitive values and infrastructure details. It also creates the risk of competing or outdated copies. A protected remote backend is more appropriate for shared infrastructure.

6. Terraform Plan and Change Review

1. Definition — What is it?

terraform plan compares the current state and real infrastructure with the desired configuration and proposes changes needed to reach the desired state.

A saved plan file allows a later apply operation to use the reviewed plan rather than generating a completely new one.

2. Purpose — Why do we use it?

- To understand proposed infrastructure changes before applying them.
- To identify resources that will be created, updated, or destroyed.
- To support approvals and change-management processes.
- To catch unintended infrastructure changes.
- To improve the audit trail for deployments.

3. Working — How does it work?

1. Terraform reads the configuration and state.
2. It refreshes or queries remote objects as required.
3. It calculates the proposed changes.
4. It displays actions such as create, update, replace, or destroy.
5. The pipeline stores the plan when needed.
6. An authorized person reviews the proposed changes before deployment.

A saved plan should be protected because it can contain sensitive data. It can also become stale if the configuration, state, or infrastructure changes before it is applied.

4. Practical Example — How do we use it in a project?

```
terraform plan -out=tfplan
terraform show tfplan
```

The first command generates a saved plan. The second displays the plan in a readable format.

In Jenkins, the plan should be generated from the intended commit and Terraform directory. The reviewer should examine the actual changes, including replacements and deletions, rather than approving only because the pipeline passed.

5. Scenario-Based Interview Questions and Answers

Q1. Your Terraform plan shows that an existing EKS cluster will be replaced. What do you do?

Answer: I stop the deployment and investigate why Terraform proposes replacement. I check changes to immutable properties, variable values, provider behavior, and state. I confirm whether replacement is intended and review the impact before requesting approval.

Q2. Why save a plan with -out=tfplan instead of running terraform apply directly?

Answer: Saving a plan separates change review from execution. The reviewer can inspect the proposed changes, and Terraform can apply that saved plan. However, the plan must be protected and applied in a compatible environment while still valid.

Q3. The saved plan fails during apply. What could be wrong?

Answer: The plan may be stale, the state may have changed, the target configuration may differ, or the environment may lack required permissions or dependencies. I would inspect the error and generate a fresh plan when necessary, then obtain any required new approval.

7. Manual Approval and Change Management

1. Definition — What is it?

A manual approval gate pauses a pipeline until an authorized person approves or rejects a proposed action.

In infrastructure CI/CD, this is especially important before production changes, destructive operations, or deployment to sensitive environments.

2. Purpose — Why do we use it?

- To prevent unreviewed infrastructure changes.
- To allow engineers to inspect a Terraform plan.
- To comply with change-management policies.
- To ensure production deployments have appropriate authorization.
- To create a controlled separation between preparation and deployment.

3. Working — How does it work?

1. Jenkins generates a plan.
2. The pipeline pauses at the approval stage.
3. The reviewer checks the plan and its expected impact.
4. An authorized person approves or rejects it.
5. Approval allows the pipeline to continue; rejection stops the deployment.

The approval step must be configured with appropriate access restrictions. A basic input step alone should not be assumed to enforce separation of duties or restrict approval to a particular group.

4. Practical Example — How do we use it in a project?

```
stage('Approval') {
    when {
        expression {
            params.ACTION == 'apply'
        }
    }
    steps {
        input message: 'Have the Terraform changes been reviewed?',
              ok: 'Approve'
    }
}
```

This is a simplified example. For production, restrict who can approve, consider a timeout and rejection handling, and make sure the approval refers to the plan generated by the intended commit.

5. Scenario-Based Interview Questions and Answers

Q1. A developer approves a plan without reviewing it. How would you improve the process?

Answer: I would provide the plan output in an accessible and protected form, require a documented change review for production, restrict approval permissions, and use a separate authorized approver when the organization's policy requires separation of duties.

Q2. What should happen if an approver rejects the plan?

Answer: The pipeline should not proceed to apply. I would record the rejection, investigate the reviewer's concerns, update the Terraform configuration if necessary, and generate a new plan before seeking approval again.

Q3. Why is approval alone not enough for a secure deployment?

Answer: Approval cannot compensate for excessive IAM permissions, exposed secrets, incorrect targeting, or unsafe configuration. I combine approval with automated validation, least privilege, protected source branches, and plan review.

8. Terraform Apply and AWS Infrastructure Deployment

1. Definition — What is it?

terraform apply executes the changes proposed by Terraform to create, update, or destroy infrastructure.

In a CI/CD pipeline, the apply stage performs the approved infrastructure deployment using the pipeline's configured AWS identity.

2. Purpose — Why do we use it?

- To provision infrastructure automatically.
- To apply reviewed configuration changes.
- To reduce manual infrastructure management.
- To maintain consistency between the declared configuration and actual resources.
- To make deployment results traceable through pipeline logs and cloud audit records.

3. Working — How does it work?

1. The pipeline reaches the apply stage after validation, planning, and any required approval.
2. Terraform uses the intended plan or generates a plan, depending on the workflow.

3. AWS checks the caller's permissions.
4. Terraform calls the necessary AWS APIs.
5. Terraform updates state as resources are created or changed.
6. Jenkins records the outcome.

When using a saved plan, terraform apply -input=false tfplan applies that plan without prompting for interactive input. The plan must be protected, and the pipeline must handle failures safely.

4. Practical Example — How do we use it in a project?

For an AWS infrastructure project:

```
terraform plan -out=tfplan
terraform apply -input=false tfplan
```

The second command is intended to run only after the saved plan has been reviewed and approved.

For your EKS project, Terraform may create or modify resources such as an EKS cluster, node group, IAM roles, security groups, and associated networking resources—depending on what is defined in your configuration.

Avoid running terraform destroy automatically in a production pipeline without explicit authorization and safeguards.

5. Scenario-Based Interview Questions and Answers

Q1. Terraform apply fails halfway through deployment. What would you do?

Answer: I would inspect the Terraform error and AWS logs, identify which resources succeeded or failed, and check the state before retrying. I would fix the root cause and generate a new plan when necessary rather than blindly rerunning or manually deleting resources.

Q2. Your Terraform apply succeeds, but the EKS cluster is not ready. Why?

Answer: Successful Terraform execution does not always mean every workload is ready. I would check EKS cluster status, node group health, IAM access, networking, and Kubernetes node readiness. Application readiness requires separate verification.

Q3. How do you make sure a production deployment uses the approved change?

Answer: I generate and review a saved plan for the intended commit, protect the plan file, restrict access to the deployment stage, and apply that saved plan. If it becomes stale or the target changes, I regenerate and review a new plan.

9. Deploying an Application to EKS

1. Definition — What is it?

Application deployment to EKS is the process of running a containerized application on a Kubernetes cluster managed by Amazon EKS.

Terraform can provision the infrastructure, while Kubernetes manifests or Helm charts typically describe how the application should run inside the cluster.

2. Purpose — Why do we use it?

- To automate application releases after infrastructure is available.
- To deploy containerized applications consistently.
- To manage replicas, service discovery, and application updates.
- To separate infrastructure provisioning from application delivery.
- To support repeatable releases and rollback strategies.

3. Working — How does it work?

A common application pipeline follows this flow:

1. Check out application code.
2. Run tests and quality checks.
3. Build a Docker image.
4. Scan the image for known vulnerabilities.
5. Push the image to Amazon ECR.
6. Authenticate to the EKS cluster with an appropriately authorized identity.
7. Deploy using Kubernetes manifests or Helm.
8. Verify rollout status and application health.

The deployment identity must have the appropriate AWS permissions and Kubernetes authorization. Access to AWS alone does not automatically grant permission to deploy workloads inside the cluster.

4. Practical Example — How do we use it in a project?

After the EKS cluster and node group are ready, a Jenkins stage might deploy an image that was already pushed to ECR:

```
stage('Deploy to EKS') {
    steps {
        bat 'aws eks update-kubeconfig --region us-east-1 --name Tetris-EKS-Cluster'
        bat 'kubectl apply -f Manifest-file/'
        bat 'kubectl rollout status deployment/tetris'
    }
}
```

This example assumes:

- The cluster exists and is reachable.
- The AWS identity can retrieve cluster connection details.
- Kubernetes authorization permits deployment.
- kubectl is installed.
- The manifest directory and Deployment name are correct.
- The manifests reference the intended image version.

In a production workflow, use immutable image tags or digests rather than relying on a mutable latest tag.

5. Scenario-Based Interview Questions and Answers

Q1. Terraform created the EKS cluster, but kubectl cannot connect. What would you check?

Answer: I would verify the cluster status, AWS identity, kubeconfig context, network reachability, endpoint access configuration, and Kubernetes access permissions. I would also confirm that the Jenkins agent can reach the cluster endpoint.

Q2. The EKS deployment stage succeeds, but the application is not accessible. What do you investigate?

Answer: I would inspect the Deployment and Pods, check container logs, events, readiness and liveness probes, Service selectors, and Ingress or load balancer configuration. I would also verify security groups and application ports.

Q3. How do you deploy a new application version safely?

Answer: I use an immutable image version, deploy through a controlled rollout strategy, verify readiness and application health, and maintain a documented rollback approach. I would not assume that a successful kubectl apply means the application is healthy.

10. Post-Deployment Verification, Monitoring, and Rollback

1. Definition — What is it?

Post-deployment verification checks whether the infrastructure or application is working correctly after deployment.

Monitoring collects information about availability, performance, resource usage, and errors. Rollback restores a previous known-good application version or reverses a change through an appropriate recovery process.

2. Purpose — Why do we use it?

- To detect deployment failures quickly.
- To confirm that services are healthy.
- To reduce downtime and SLA impact.
- To identify problems that automated deployment commands cannot detect.
- To support reliable recovery when a release fails.

3. Working — How does it work?

After deployment, the pipeline or monitoring system can:

1. Check AWS resource status.
2. Verify EKS node and Pod readiness.
3. Check Kubernetes rollout status.
4. Run a health-check request against the application.
5. Send notifications for success or failure.
6. Trigger an investigation or recovery procedure if health checks fail.

Tools can include CloudWatch, Prometheus, Grafana, and centralized application logs.

Rollback depends on what changed. An application image can often be rolled back to a previous version. Infrastructure changes, database migrations, and stateful resources may require a different recovery plan.

4. Practical Example — How do we use it in a project?

For an EKS deployment, use:

```
kubectl get nodes
kubectl get pods -A
kubectl rollout status deployment/tetris
kubectl describe deployment tetris
```

If a Deployment rollout fails, investigate its events and logs before deciding on recovery.

For an application Deployment where the previous ReplicaSet is still available, a rollback may be possible with:

```
kubectl rollout undo deployment/tetris
```

This is not a universal rollback for Terraform-managed infrastructure or database changes. Confirm the target Deployment, previous revision, and impact before executing it.

5. Scenario-Based Interview Questions and Answers

Q1. Jenkins reports a successful deployment, but users cannot access the application. What do you do?

Answer: I verify Pod readiness, application logs, Kubernetes Services, Ingress or load balancer health, networking, DNS, and application health checks. I would communicate the impact and use the established recovery procedure if the service cannot be restored quickly.

Q2. How do you monitor an EKS application in production?

Answer: I combine Kubernetes health checks, application logs, CloudWatch metrics and logs, and Prometheus/Grafana metrics where available. I configure meaningful alerts for availability, error rates, latency, and resource pressure.

Q3. Can terraform destroy be used as a rollback?

Answer: Not as a general rollback strategy. Destroy removes managed resources and can cause significant outages or data loss. I would use a deliberate recovery plan, which might include reverting configuration, applying a reviewed Terraform plan, restoring data, or rolling back an application release.

11. Environment Separation and Deployment Parameters

1. Definition — What is it?

Environment separation means maintaining distinct configurations and access controls for environments such as development, testing, staging, and production.

Pipeline parameters allow users to supply controlled inputs, such as an action or deployment target.

2. Purpose — Why do we use it?

- To test changes without affecting production.
- To isolate cloud resources and credentials.
- To control which environments a pipeline can modify.
- To make deployment targets explicit.
- To reduce the risk of applying changes to the wrong AWS account or region.

3. Working — How does it work?

A pipeline determines its target environment from approved configuration, branch rules, credentials, and controlled parameters.

Typical separation includes:

- Different AWS accounts or roles.
- Different Terraform state keys or workspaces where appropriate.
- Environment-specific variables.
- Restricted production deployment jobs.
- Separate approval and audit requirements.

A parameter alone is not a security boundary. A user who can select production must still be prevented from deploying there unless they are authorized.

4. Practical Example — How do we use it in a project?

A simple parameter declaration is:

```
parameters {
    choice(
        name: 'ENVIRONMENT',
        choices: ['dev', 'stage', 'prod'],
        description: 'Select the target environment'
    )
}
```

The pipeline must map each permitted value to the correct configuration, AWS identity, and state backend. Production should use stricter access controls and approval requirements.

For Terraform, avoid accidentally sharing the same state key between unrelated environments.

5. Scenario-Based Interview Questions and Answers

Q1. A developer selects prod in the Jenkins parameter, but they should only deploy to development. What prevents the deployment?

Answer: I would enforce authorization through protected deployment jobs, role-based permissions, and environment-specific credentials. I would not rely on hiding the production option or checking only a user-supplied parameter.

Q2. Why separate Terraform state between development and production?

Answer: It prevents unrelated environments from managing the same state, reduces the risk of accidental changes, and allows different access controls and change-management processes.

Q3. The pipeline uses the wrong AWS region. How do you prevent this?

Answer: I explicitly configure the intended region, verify the AWS account and identity before planning, and review the target environment in pipeline logs. I also use controlled environment-specific configuration rather than relying on a developer's local AWS defaults.

12. End-to-End Jenkins CI/CD Pipeline Example

This example brings together the major stages for a Terraform infrastructure deployment.

It is a learning template for a Windows Jenkins agent, not a complete production security implementation. It assumes AWS authentication, repository configuration, Terraform backend access, approval authorization, and workspace management have already been configured.

```
pipeline {
    agent { label 'windows-terraform' }

    options {
        timestamps()
        disableConcurrentBuilds()
    }

    parameters {
        choice(
            name: 'ACTION',
            choices: ['plan', 'apply'],
            description: 'Select the Terraform action'
        )
    }

    environment {
        AWS_DEFAULT_REGION = 'us-east-1'
        TF_IN_AUTOMATION = 'true'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
                bat 'git rev-parse HEAD'
            }
        }

        stage('Terraform Init') {
            steps {
                dir('EKS-TF') {
                    bat 'terraform init -input=false'
                }
            }
        }

        stage('Validate') {
            steps {
                dir('EKS-TF') {
                    bat 'terraform fmt -check -recursive'
                    bat 'terraform validate'
```

```
                }
            }
        }

        stage('Verify AWS Identity') {
            steps {
                bat 'aws sts get-caller-identity'
            }
        }

        stage('Plan') {
            steps {
                dir('EKS-TF') {
                    bat 'terraform plan -input=false -out=tfplan'
                    bat 'terraform show tfplan'
                }
            }
        }

        stage('Approval') {
            when {
                expression {
                    params.ACTION == 'apply'
                }
            }
            steps {
                input message: 'Review and approve the Terraform plan?',
                      ok: 'Approve'
            }
        }

        stage('Apply') {
            when {
                expression {
                    params.ACTION == 'apply'
                }
            }
            steps {
                dir('EKS-TF') {
                    bat 'terraform apply -input=false tfplan'
                }
            }
        }

        stage('Verify EKS') {
            when {
                expression {
                    params.ACTION == 'apply'
                }
            }
            steps {
                bat 'aws eks describe-cluster --name Tetris-EKS-Cluster --region %AWS_DEFAULT_REGION%
                    --query cluster.status --output text'
            }
        }
    }

    post {
        success {
            echo 'CI/CD pipeline completed successfully.'
        }

        failure {
            echo 'Pipeline failed. Review the failed stage and logs.'
        }

        always {
            echo 'Pipeline execution finished.'
        }
    }
}
```

Important limitations to fix before production

- The example assumes the repository root contains EKS-TF. Adjust the directory if the Jenkins workspace structure differs.
- Configure AWS authentication securely; the pipeline does not create or attach an IAM role.
- Protect the saved tfplan file and Terraform state. The saved plan should not be exposed to unauthorized users.
- Restrict approval permissions and bind the approval to the intended reviewed changes.
- The EKS verification stage checks cluster status, not whether the application is healthy.
- For production, use a controlled process to ensure the approved plan remains the plan being applied, and regenerate approval when the plan becomes stale.
- Add tests, security scans, failure notifications, and a documented recovery procedure according to project requirements.

13. Common Real-Time CI/CD Failures

Problem	Likely cause	First troubleshooting step
Jenkins does not trigger	Webhook or branch configuration issue	Check webhook delivery and job configuration
Git checkout fails	Authentication or repository URL issue	Verify repository access and credential scope
terraform init fails	Backend, provider, or network issue	Inspect the first initialization error
terraform validate fails	Invalid configuration or missing required input	Read the validation message and inspect the affected file
AWS authentication fails	Missing or conflicting credentials	Run aws sts get-caller-identity in the same agent context
AWS AccessDenied	Insufficient permissions or explicit deny	Identify the denied action and inspect applicable policies
Terraform plan proposes deletion	Configuration drift or changed resource attributes	Stop and review the plan before applying
Terraform state lock error	Another operation holds a lock or stale lock exists	Identify the owner before considering a lock recovery
EKS cluster is inaccessible	Endpoint, network, or authorization problem	Verify cluster status, kubeconfig, connectivity, and access
Application rollout fails	Image, configuration, scheduling, or health issue	Inspect Pods, events, logs, and rollout status

Interview tip: Explain how you isolate the failing stage, collect evidence, identify the root cause, apply the smallest safe fix, and verify the result. Avoid saying that you simply rerun the pipeline without understanding the failure.

14. Advanced Scenario-Based Interview Questions

Q1. A developer pushes a Terraform change that accidentally exposes an EKS cluster endpoint publicly. What do you do?

Answer: I would stop the deployment before apply, if it has not yet happened. I would review the plan and network access configuration, determine whether the exposure is intended, and correct the configuration or access restrictions. If already deployed, I would follow the incident procedure to restrict access and verify the fix.

Q2. Your pipeline creates an EKS cluster, but the node group fails to join the cluster. What would you investigate?

Answer: I would check node group status, EC2 instance health, IAM role permissions, subnet routing, security group rules, cluster endpoint access, and node bootstrap or networking errors. I would use AWS and Kubernetes diagnostics to isolate whether the problem is IAM, networking, or node configuration.

Q3. A Terraform pipeline failed after creating some AWS resources. How do you recover?

Answer: I would inspect the failure, Terraform state, and the actual AWS resources. I would correct the root cause and generate a new plan to understand the remaining changes. I would not manually delete resources or alter state without first evaluating the consequences.

Q4. How do you prevent two different teams from deploying conflicting changes?

Answer: I would use remote state with locking, prevent overlapping builds where appropriate, establish clear ownership of infrastructure, and use controlled deployment windows or separate state boundaries when necessary.

Q5. What is the difference between continuous delivery and continuous deployment?

Answer: Continuous delivery automates changes up to a release-ready state, usually with a manual decision before production release. Continuous deployment automatically releases changes to production after all configured checks pass. The chosen approach depends on business risk and approval requirements.

Q6. How would you secure a Jenkins pipeline that deploys Terraform to AWS?

Answer: I would use least-privilege IAM roles and temporary credentials, restrict Jenkins permissions, protect credentials and state, scan the configuration, review the Terraform plan, enforce production approvals, protect source branches, and audit deployments.

Q7. What happens if a GitHub commit changes after a plan has been approved?

Answer: I would not blindly apply the old approval to the new code. I would ensure the plan corresponds to the intended commit. If the configuration or plan changes, I would generate and review the new plan and obtain approval again when required.

Q8. How do you demonstrate that the deployment succeeded?

Answer: I would provide the Jenkins build result, commit hash, Terraform plan and apply outcome, AWS resource status, EKS node and workload readiness, and application health-check results. A green Jenkins status alone is not sufficient proof of application health.

15. Final Revision Checklist

- [ ] Explain the GitHub -> Jenkins -> Terraform -> AWS/EKS architecture.
- [ ] Explain how GitHub webhooks trigger Jenkins.
- [ ] Write a Jenkinsfile with Checkout, Init, Validate and Plan stages.
- [ ] Explain remote Terraform state and state locking.
- [ ] Explain AWS authentication and IAM roles.
- [ ] Interpret a Terraform plan and identify destructive changes.
- [ ] Implement an appropriate approval gate.
- [ ] Explain safe Terraform apply and recovery.
- [ ] Verify EKS cluster, node group, and application health.
- [ ] Explain least privilege, secrets handling, and production access controls.

Best final interview answer to remember:

“My CI/CD pipeline integrates GitHub, Jenkins, Terraform, and AWS. GitHub triggers Jenkins when code changes. Jenkins checks out the correct commit, initializes Terraform, runs formatting and validation, and generates a plan. After reviewing the proposed changes and receiving the required approval, Jenkins applies the saved plan using a securely configured AWS identity. For EKS, I verify the cluster and node group and, when application deployment is included, verify Kubernetes rollout and application health. I protect credentials and state, follow least privilege, and investigate failures before retrying or rolling back.”